on which Memcached listens; and in */etc/memcached.conf*, look for -m 64 and change it to -m 256 to allow 256 MB RAM for Memcached.

Start the Memcached service, restart Apache and install the Drupal Memcached module to allow Drupal to use Memcached; specify the following in your *settings.php* file:

```
$conf['cache_backends'][]= 'sites/all/modules/memcache/
memcache.inc';
$conf['cache_default_class'] = 'MemCacheDrupal';
$conf['memcache_key_prefix'] = 'unique_string';
Memcached will cache DB queries and provide a significant
performance improvement for Drupal sites.
```

Configure MySQL for optimum performance

Some of our installations have a dedicated Virtual Private Server for the database. If you have a server dedicated to MySQL and it has a memory of 1 GB, you can start with the following values in your configuration file *(/etc/mysql/my.cnf)*.

```
key_buffer = 256M
query_cache_size = 128MB
query_cache_limit = 4MB
table_cache = 512
sort_buffer_size = 32M
myisam_sort_buffer_size = 32M
tmp_table_size = 64MB
delay_key_write = 1
wait_timeout = 60
```

The possibilities of improving database (DB) performance are endless and there is no one magic solution. Rather than embarking on an elaborate DB tuning exercise, I believe the biggest performance improvements come from identifying and tuning your slow DB queries. Activate the slow query log and identify the slow queries and rewrite them. You should do this especially if you have installed the *views* module. You could also use *Devel* to identify the queries that take a lot of time. Having identified the slow queries, change them to make them execute faster.

Check your code, and follow the best practices in your modules

Finally, if you have written custom modules then use the *Devel* module to identify slow queries and rewrite them. Use the XHProf module (along with the XHProf php extension) to profile memory and CPU cycles, and refactor your code.

In the book 'High Performance Websites', Steve Sounders mentions some rules to speed up your website. Keep them in mind while writing your modules. Ensure that you don't embed PHP code, CSS and JavaScript in your content pages.

Figure 4: Pingdom3

Figure 5: Pingdom4

The performance techniques described in this article are sufficient for most Drupal installations.

HTTP caching using Apache

Caching frequently accessed content at the browser or proxy can improve response times because the content will not be retrieved from the server. This saves bandwidth, decreases the load on the server and reduces latency. Enable the *mod_expires* and *mod_headers* modules. To enable caching at the server side, enable the *mod_cache* module. You could also use Google's *mod_pagespeed*.

Separate servers for static and dynamic content

Apache processes serving Drupal sites are between 40 MB to 60 MB. Their size grows to accommodate the content being served and doesn't decrease till the process dies. Let's assume that your home page consists of several images and the browser makes multiple requests to retrieve them. Assuming that we have the *KeepAlive* directive set to 'On', each request is serviced by an Apache process that is 60 MB in size, even though the request is for an image that can be served by a 1 to 3 MB process.

You could set up a 'Baby Apache' compiled with just the minimum required modules to serve static content like images. The Baby Apache will receive all client requests. It should handle requests for images and it should forward all other requests to the heavyweight Apache that is serving the dynamic content. You will need to enable the *mod_proxy* and *mod_rewrite* modules, and set up the rewrite rules in your configuration file.

Using a content delivery network (CDN)

We use CDNs sparingly. Consider a CDN only if your site has a reasonable number of visitors from a region. Do not include a CDN in your stack just to increase the throttling of your site; the added maintenance is not